

REPUBLIQUE DU CAMEROUN

Paix-Travail-Patrie

MINISTERE DE L'ENSEIGNEMENT SUPERIEUR

Université de Yaoundé 1

Institut Saint Jean

REPUBLIC OF CAMEROON

Peace-Work-Fatherland

MINISTERE DE L'ENSEIGNEMENT SUPERIEUR

Université de Yaoundé 1

Institut Saint Jean



DOSSIER DE REALISATION

THEME

***CONCEPTION ET MISE EN ŒUVRE D'UN WEB
USER MONITOR :Cas ARCH LINUX***

MASTER I

Option : SECURITE DES SYSTEMES D'INFORMATIONS

Rédigé et présenté par :

- ASSOUMOU ASSOUMOU Edwin Alahn
- NANGOU POUNTOUGNIGNI Marilyn

Sous la supervision de :

M. NGUIMBUS Emmanuel

Année Académique
2025-2026

Table des matières

INTRODUCTION ET ENVIRONNEMENT DE REALISATION	V
1. Justification de la Démarche d'Ingénierie	V
2. Pile Technologique et Écosystème Logiciel.....	V
3. Infrastructure de Développement et Cible de Déploiement	V
I. ARCHITECTURE DE LA SOLUTION	VI
1. Schématisation de l'Architecture Globale	VI
2. Analyse Multicouche du Système	VI
3. Flux de Données et Transition Kernel-to-User Space	VII
4. Séparation des Responsabilités	VII
II. INGENIERIE BACKEND	VIII
1. Parsing Binaire et Désérialisation de la Structure utmp.....	VIII
2. Algorithme de Corrélation et Logique de "Data Joining"	VIII
3. Monitoring des Ressources via l'Interface VFS (procfs).....	VIII
4. Signalisation et Gestion des Processus Orphelins.....	IX
III. CHARTE GRAPHIQUE ET DESIGN UI/UX	IX
1. Identité Visuelle et Psychologie Chromatique	IX
2. Architecture de l'Information et Ergonomie (UX)	IX
IV. GALERIE DES INTERFACES	X
1. Gestionnaire Dynamique des Sessions Utilisateurs	X
a. Analyse détaillée des repères de l'interface :	X
V. DEVELOPPMENT FRONTEND	XII
1. Architecture Logicielle et Découplage (Clean Arch).....	XII
2. Réactivité Granulaire via les Angular Signals.....	XII
3. Communication Asynchrone et Flux de Données (RxJS).....	XIII
4. Design System et Intégration Tailwind CSS	XIII
VI. TESTS ET VALIDATION	XIV
1. Protocole de Recette Fonctionnelle.....	XIV
CONCLUSION	XV
a. Synthèse des Réalisations	XV
PERSPECTIVES D'ÉVOLUTION	XV

LISTE DES FIGURES

INTRODUCTION ET ENVIRONNEMENT DE REALISATION

La phase de réalisation constitue l'aboutissement technique du cycle de développement. Elle transpose les modèles théoriques de la phase de conception en une solution logicielle fonctionnelle, interopérable et sécurisée. Ce dossier documente les choix d'implémentation "As-Built" (tels que réalisés) et les spécificités de l'environnement de déploiement.

1. Justification de la Démarche d'Ingénierie

Le développement de la solution **ALWUM(ARCH LINUX Web User Monitor)** a été guidé par trois piliers fondamentaux propres aux systèmes critiques :

- **La Fidélité des Données** : L'extraction doit refléter l'état réel du noyau Linux sans latence d'interprétation.
- **L'Isolation (Sandboxing)** : La séparation stricte entre la logique d'acquisition (Backend) et la logique de présentation (Frontend) pour limiter la surface d'attaque.
- **La Performance Non-Bloquante** : L'utilisation de paradigmes asynchrones pour garantir la disponibilité de l'interface même lors de pics de charge système.

2. Pile Technologique et Écosystème Logiciel

Le choix de la stack technique ne repose pas sur une préférence de développement, mais sur une adéquation aux contraintes du système cible (**Arch Linux**) :

- **Moteur d'Exécution Backend (FastAPI / Python 3.12)** : Contrairement aux frameworks synchrones, l'architecture de FastAPI permet une gestion concurrente des requêtes HTTP et des tâches de fond. **Python a été choisi pour ses bibliothèques de bas niveau (struct, os, pwd)** facilitant l'interfaçage avec les structures C du noyau.
- **Interface de Pilotage (Angular 19 / TypeScript)** : Angular 19 a été sélectionné pour sa robustesse typée et l'introduction des **Signals**. Cette technologie permet une mise à jour granulaire du DOM (Document Object Model), essentielle pour un tableau de bord de monitoring.
- **Communication et Sérialisation** : Le protocole **RESTful** sur HTTP/S avec sérialisation **JSON** a été retenu pour assurer une neutralité totale entre le format de données brut du noyau (binaire/texte) et la couche de présentation.

3. Infrastructure de Développement et Cible de Déploiement

La réalisation s'est appuyée sur un environnement de virtualisation simulant un déploiement en centre de données :

- **Hôte de Développement** : Station de travail **HP EliteBook 840 G5**, optimisée pour la virtualisation matérielle.
- **Système Cible (Cible de Monitoring)** : Instance **Arch Linux** (Rolling Release), choisie pour sa proximité avec le code source original du noyau Linux (Vanille) et l'absence de couches d'abstraction superflues.
- **Architecture Réseau** : Mise en place d'un réseau **Host-Only**. Cette topologie crée un segment réseau isolé, protégeant les flux de monitoring des interceptions externes tout en permettant une communication bidirectionnelle entre l'interface de gestion (Windows) et le serveur de données (Arch Linux).

I. ARCHITECTURE DE LA SOLUTION

L'architecture du système est structurée selon un modèle de **conception en couches (Layered Architecture)**. Cette approche permet d'isoler la complexité de l'interaction noyau de la logique de présentation, garantissant ainsi une maintenance facilitée et une robustesse face aux évolutions du Kernel Linux.

1. Schématisation de l'Architecture Globale

Le schéma suivant synthétise l'interaction entre les différentes couches de la solution :

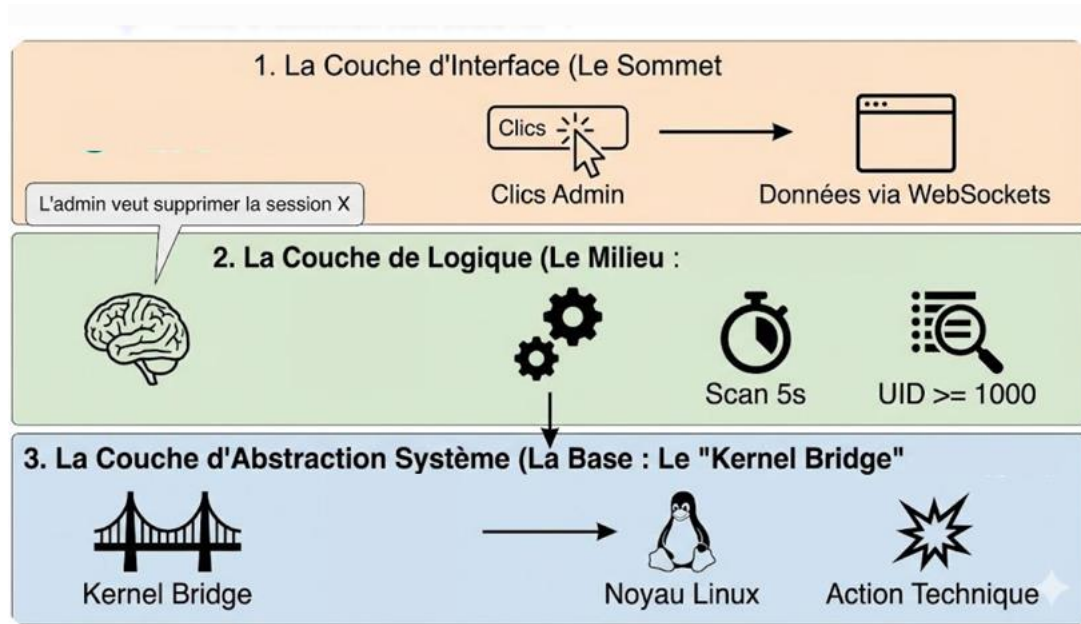


Figure 1 : Architecture du système

2. Analyse Multicouche du Système

L'implémentation repose sur une hiérarchie descendante, du sommet (l'utilisateur) vers la base (le matériel) :

- **La Couche d'Interface** : C'est l'unique point de contact pour l'administrateur développé sous Angular. Bien que l'architecture cible à terme l'usage de WebSockets pour une réactivité instantanée, la version actuelle implémente un mécanisme de **Polling HTTP asynchrone** (intervalle de 5s), offrant un compromis optimal entre fraîcheur des données et consommation de bande passante.
- **La Couche de Logique** : Située dans le Backend FastAPI, cette couche orchestre le flux d'informations. Elle assure le filtrage critique pour isoler les utilisateurs réels des processus système.
- **La Couche d'Abstraction Système (Le kernel Bridge)** : Il s'agit du pont direct avec le noyau Linux. Cette couche traduit les demandes logiques en **Appels Système (Syscalls)**.

3. Flux de Données et Transition Kernel-to-User Space

Le passage de l'information entre le noyau et l'interface suit un protocole de transformation rigoureux :

- **Extraction (Raw Data)** : Le "Kernel Bridge" lit les fichiers binaires. À ce stade, les données sont brutes, souvent sous forme de flux d'octets ou de buffers non structurés.
- **Structuration (Parsing)** : La logique Backend décode ces octets (via le module Python struct) pour recréer des objets manipulables.
- **Sérialisation (JSON)** : Les données structurées sont converties en objets JSON. Ce format universel permet de traverser le réseau (via le tunnel Host-Only) sans perte d'intégrité jusqu'au Frontend Angular.

4. Séparation des Responsabilités

L'indépendance des couches garantit que toute modification du noyau (ex: mise à jour de la structure utmp dans une future version d'Arch Linux) n'impacte que la **Couche d'Abstraction**, laissant l'interface Angular intacte.

II. INGENIERIE BACKEND

Le backend développé avec le framework **FastAPI**, assure la transition critique entre l'espace noyau (*Kernel-Space*) et l'espace utilisateur (*User-Space*). Cette section détaille les mécanismes d'extraction et de structuration des données volatiles du noyau.

1. Parsing Binaire et Désérialisation de la Structure utmp

L'interfaçage avec le fichier de comptabilité système `/var/run/utmp` constitue le pivot technique du projet. Contrairement aux flux textuels du système de fichiers virtuel `/proc`, `utmp` impose une lecture binaire synchrone alignée sur la structure C standard définie dans `utmp.h`.

- **Alignement et Masquage Binaire** : L'implémentation repose sur le module `struct` de Python, agissant comme un compilateur de formats. Nous avons défini un masque de dépaquetage (*Unpacking Mask*) respectant strictement l'alignement des octets de la structure C.
 - *Spécification technique* : Le typage inclut des entiers courts (`short`) pour `ut_type` et des tableaux de caractères à taille fixe pour `ut_user` (32 octets) et `ut_line` (32 octets).
- **Filtrage par État de Processus** : Une analyse heuristique est appliquée sur le champ `ut_type`. Seules les entrées marquées `USER_PROCESS` (valeur 7) sont extraites, garantissant l'exclusion des processus d'initialisation (`INIT_PROCESS`) ou de terminaison (`DEAD_PROCESS`), évitant ainsi tout faux positif dans le monitoring des sessions actives.

2. Algorithme de Corrélation et Logique de "Data Joining"

Pour garantir l'intégrité des données affichées, le backend implémente un algorithme de corrélation croisée entre les sources d'information statiques et volatiles :

- **Référentiel d'Identité (Source : `/etc/passwd`)** : Le système interroge `/etc/passwd` via le module `pwd` pour récupérer la liste exhaustive des utilisateurs.
- **Ségrégation des Privilèges (UID Filter)** : Un prédicat de sécurité est appliqué pour isoler les identités à haut privilège et les comptes de service. Pour éviter de polluer l'interface avec les comptes de service (`bin`, `daemon`, etc.), un filtre est appliqué sur l'User ID. Seuls les comptes "humains" sont transmis au moteur d'analyse.
- **Calcul de l'État de Présence** : Une jointure logique est opérée entre la liste des utilisateurs filtrés et les descripteurs de session extraits de `utmp`. L'absence de correspondance pour un UID donné définit l'état **Idle**, tandis qu'une occurrence valide définit l'état **Active**.

3. Monitoring des Ressources via l'Interface VFS (`procfs`)

L'acquisition des métriques de performance s'appuie sur le **Virtual File System (VFS)**, offrant une interface normalisée vers les compteurs du noyau :

- **Analyse de la Volatilité Mémoire (`/proc/meminfo`)** : L'algorithme ne se limite pas à la lecture de la mémoire libre. Il calcule la mémoire "réellement disponible" en agrégeant les buffers et le cache paginé, fournissant ainsi une métrique de pression mémoire fidèle à la gestion algorithmique du noyau Linux.

- **Vecteurs de Charge (/proc/loadavg) :** L'extraction du *Load Average* permet d'analyser la saturation de la file d'attente de l'ordonnanceur. Ces vecteurs (1, 5, 15 min) sont essentiels pour diagnostiquer des goulots d'étranglement CPU avant qu'ils n'impactent la réactivité des sessions utilisateurs.

4. Signalisation et Gestion des Processus Orphelins

La fonction d'administration (éjection d'un utilisateur) repose sur l'appel système `kill(2)`.

- **Validation des Cibles :** Avant toute émission de signal, le backend vérifie l'existence du PID via le répertoire `/proc/[pid]`.
- **Stratégie de Signalisation :** L'envoi du signal `SIGKILL (9)` est réservé aux actions d'éjection forcée, assurant une libération immédiate des ressources allouées à la session cible.
- **Garde-fou Algorithmique :** Une liste blanche de PIDs protégés (notamment le PID 1 et les processus dont le PPID appartient à la hiérarchie système)

III. CHARTE GRAPHIQUE ET DESIGN UI/UX

Le design de l'interface n'est pas le fruit d'une intention purement décorative, mais répond à des impératifs d'efficacité cognitive. L'objectif est de minimiser la charge mentale de l'administrateur système lors de l'analyse de données volatiles.

1. Identité Visuelle et Psychologie Chromatique

L'interface adopte une charte graphique "**Dark Mode**" (Thème sombre), standard industriel pour les outils de supervision technique.

- **Contraste et Lisibilité :** L'utilisation de fonds sombres combinés à une typographie claire réduit la fatigue oculaire lors d'une surveillance prolongée.
- **Sémantique des Couleurs (Signaling) :** Un code couleur normé est appliqué pour une interprétation instantanée de l'état du système :
 - **Émeraude (#10b9813) :** État nominal ou session utilisateur active.
 - **Ambre (#f59e0b) :** Charge système élevée ou latence de réponse.
 - **Rubis (#ef4444\$) :** Action destructive (suppression de processus) ou erreur de communication API.

2. Architecture de l'Information et Ergonomie (UX)

Le layout suit une hiérarchie "**Top-Down Monitoring**" :

1. **Niveau Macro (Dashboard) :** Les indicateurs de santé globale (CPU/RAM) sont placés en zone supérieure gauche, point d'entrée naturel de la lecture oculaire.
2. **Niveau Méso (Liste des Sessions) :** Le corps central présente la table des sessions, structurée pour permettre un balayage rapide des noms d'utilisateurs et de leurs terminaux respectifs.
3. **Niveau Micro (Actions) :** Les boutons d'administration sont isolés pour prévenir les clics accidentels, avec une demande de confirmation logicielle pour toute action irréversible sur un processus.

IV. GALERIE DES INTERFACES

Cette section présente les artefacts visuels de la solution finale, validant la conformité entre les spécifications fonctionnelles et la réalisation logicielle.

1. Gestionnaire Dynamique des Sessions Utilisateurs

L'interface présentée ci-dessous est le point d'orgue du projet. Elle opère la synthèse visuelle de la jointure binaire entre les structures utmp et le référentiel /etc/passwd.

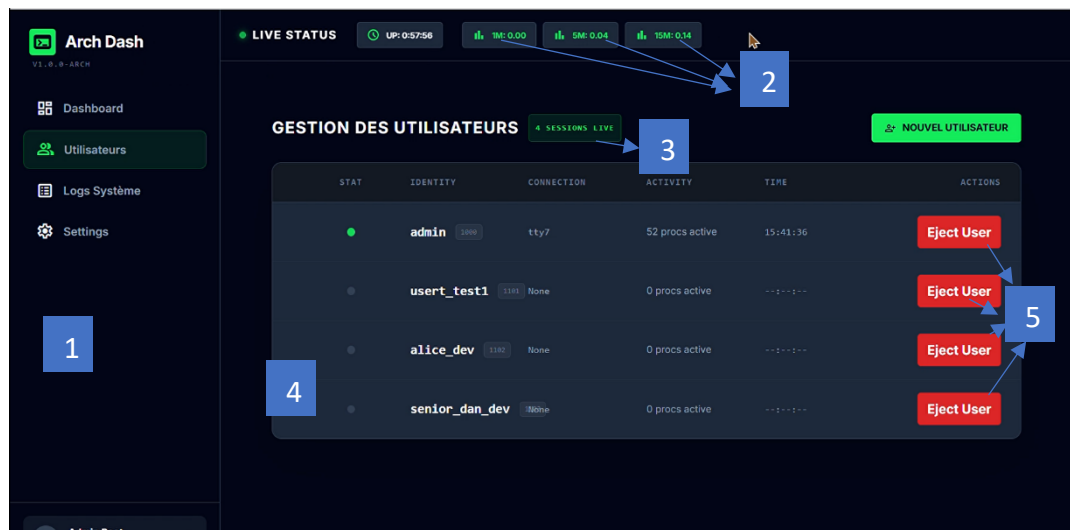


Figure 2 : Interface de monitoring

- **Visualisation des sessions** : Chaque ligne de la table représente une session interactive unique (TTY ou PTS).
- **Interactivité** : L'administrateur dispose d'une commande d'éviction permettant l'envoi d'un signal atomique au backend.

a. Analyse détaillée des repères de l'interface :

- **REPERE [1] - Navigation et Contexte Applicatif** : L'activation du module "Utilisateurs" déclenche l'instanciation du composant Angular dédié, lequel initie le cycle de polling vers l'API FastAPI.
- **REPERE [2] - Indicateurs de Vitalité et Charge Système** : Situés en en-tête, ces indicateurs affichent l'Uptime et le Load Average (1M, 5M, 15M). Ces données, extraites via procsfs, permettent à l'administrateur d'évaluer la charge de l'ordonnanceur avant toute action d'éjection forcée.
- **REPERE [3] - Compteur d'État en Temps Réel** : Le badge "4 SESSIONS LIVE" illustre la réactivité des Angular Signals. Il reflète le décompte exact des entrées de type USER_PROCESS identifiées lors du dernier scan atomique du fichier /var/run/utmp.
- **REPERE [4] - Table de Monitoring Granulaire** : Chaque ligne de cette grille représente une session interactive unique.

- **Identity** : Résolution du nom d'utilisateur et affichage de l'UID (≥ 1000).
- **Connection** : Identification du terminal de contrôle (ex: tty7).
- **Activity** : Monitoring des processus enfants rattachés à la session.
- **REPERE [5] - Commandes d'Éviction (Actions)** : Les boutons "Eject User" constituent l'interface de contrôle du **Syscall kill(2)**. L'utilisation de la couleur rouge (Rubis) sert de garde-fou visuel, signalant le caractère irréversible de l'arrêt du processus de session.

V. DEVELOPPMENT FRONTEND

Le développement de la couche de présentation a été guidé par les principes de la **Clean Architecture**, visant à maximiser la maintenabilité et à isoler les règles de gestion des spécificités du framework.

1. Architecture Logicielle et Découplage (Clean Arch)

L'architecture logicielle implémentée repose sur une segmentation stricte des responsabilités (SOC - *Separation of Concerns*) :

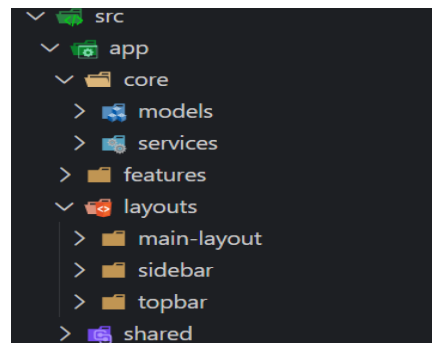


Figure 3 : Architecture Front-end

- **Le Noyau (Module Core) :**
 - **models/ :** Contient les entités métier et les interfaces de données. C'est ici que sont définis les contrats de données (ex: UserSession, SystemMetrics) qui servent de référence à toute l'application.
 - **services/ :** Implémentation des adaptateurs d'infrastructure. Ces services encapsulent la logique de communication avec l'API FastAPI (via HttpClient) et la gestion des flux RxJS.
- **La Couche Domaine (Features/)** : Ce répertoire regroupe les modules fonctionnels de l'application (ex: Dashboard, Gestion des utilisateurs). Chaque feature est autonome, facilitant les tests et l'évolution indépendante des fonctionnalités.
- **L'Architecture de Présentation (Layouts/)** : permet une séparation des composants structurels tels que la barre de navigation et la barre latérale (sidebar, topbar) du contenu principal (main-layout). Cette approche favorise la réutilisation des composants et une gestion centralisée du design system.

2. Réactivité Granulaire via les Angular Signals

Pour un monitoring temps réel, la performance de rendu est critique. Nous avons substitué la détection de changement classique par les **Signals** :

- **Effect & Computed :** Le calcul du pourcentage d'utilisation RAM est encapsulé dans un computed signal, qui ne se recalcule que lorsque la valeur brute reçue du backend change.

- **Performance** : Cette approche réduit la charge CPU sur la machine hôte (HP EliteBook), car Angular ne rafraîchit que les nœuds précis du DOM impactés par la mise à jour système.

3. Communication Asynchrone et Flux de Données (RxJS)

La communication avec le backend Arch Linux est régie par des flux d'observables :

- **Polling Stratégique** : Utilisation de l'opérateur interval de RxJS combiné à switchMap. Cette combinaison garantit qu'une nouvelle requête de monitoring n'est lancée que si la précédente est terminée, évitant ainsi l'empilement de requêtes en cas de latence réseau.
- **Error Handling** : Un intercepteur global capture les codes d'état HTTP. Les erreurs 403 (Permissions refusées pour le kill) ou 504 (Gateway Timeout) sont transformées en notifications sémantiques pour l'administrateur.

4. Design System et Intégration Tailwind CSS

L'interface repose sur **Tailwind CSS**, permettant une stylisation utilitaire rapide et un respect strict de la charte graphique définie.

VI. TESTS ET VALIDATION

Cette phase de recette technique valide la conformité du logiciel **Arch Dash** face aux exigences de stabilité et de sécurité définies dans le cahier des charges.

1. Protocole de Recette Fonctionnelle

Le système a été soumis à une batterie de tests d'intégration simulant des conditions réelles d'exploitation :

- **Test d'Instanciation de Session** : Ouverture d'une session SSH distante sur la cible Arch Linux.
 - *Résultat attendu* : Mise à jour du signal Angular dans un délai de 5s. Extraction correcte de l'identifiant de terminal (pts/x).
- **Test d'Éjection Atomique (Kill)** : Déclenchement d'un signal SIGKILL depuis l'interface pour un utilisateur standard.
 - *Résultat attendu* : Fermeture immédiate de la socket SSH et libération des descripteurs de fichiers associés. Retour d'état 200 OK au frontend.

CONCLUSION

Le projet constitue une preuve de concept (PoC) aboutie d'un système de supervision hybride. Il démontre qu'il est possible de concilier la rudesse des appels système avec la modernité et l'élégance des frameworks web contemporains.

a. Synthèse des Réalisations

Ce travail a permis de valider une approche d'ingénierie logicielle complète :

- **Maîtrise du Bas-Niveau** : Capacité à interfacier des langages de haut niveau (Python/TypeScript) avec les structures de données binaires du noyau Linux.
- **Rigueur Architecturale** : Application des principes de la **Clean Architecture**, garantissant une séparation stricte entre les modèles métier et les détails d'implémentation.
- **Expérience Utilisateur Technique** : Conception d'une interface facilitant la conscience situationnelle pour un administrateur système.

PERSPECTIVES D'ÉVOLUTION

L'architecture modulaire retenue permet d'envisager plusieurs axes d'amélioration pour un passage en production industrielle :

- **Temps Réel Absolu** : Migration du polling HTTP vers un protocole **Full-Duplex via WebSockets**, permettant un "push" instantané des événements noyau vers l'administrateur.
- **Audit et Journalisation** : Intégration d'un module de log centralisé (via journald) pour tracer chaque action d'éjection à des fins d'audit de sécurité.
- **Contrôle d'Accès (RBAC)** : Implémentation d'une couche d'authentification forte pour restreindre les privilèges d'administration selon le profil de l'utilisateur connecté sur l'interface.

En conclusion, la réalisation de ce système de monitoring valide non seulement des compétences techniques transverses, mais aussi une capacité à concevoir des outils d'administration sécurisés, performants et évolutifs.